SYSTEM ARCHITECTURE & TECHNICAL MANUAL

AUTONOMOUS AI EXECUTIVE

A.U.R.I.X

Autonomous Universal Reasoning & Interaction Executive

A Fully Autonomous, Edge-Governed, Air-Gapped Desktop AI Agent

Zero Network Sockets | 100% On-Device Execution | Bare-Metal Hardware Governor | Continuous QLoRA Learning | Autonomous Self-Healing | PyO3 In-Process FFI

SYSTEM VERSION v0.1.0 Core Engine	CORE SYSTEMS STACK Rust 2021 + PyO3 FFI	AI INFERENCE & LORA Qwen 3:4B (4-Bit NF4) + Unsloth
DESKTOP INTERFACE Slint 1.8 Native GUI	AUDIO SUBSYSTEM Whisper.cpp (STT) + Piper (TTS)	PERSISTENCE & ANALYTICS SQLite + DuckDB + ChromaDB

AURIX Engineering Team • Complete Architectural & Operational Guide

Target Hardware Profile: 16 GB Host RAM (12 GB Ceiling) • NVIDIA RTX 4060 8 GB VRAM (6 GB Ceiling)

CONFIDENTIAL • ENGINEERING SPEC

1 Executive Summary & System Invariants

A.U.R.I.X (Autonomous Universal Reasoning & Interaction Executive) is a cutting-edge, locally executing autonomous desktop AI agent. Engineered to operate completely on-device without cloud telemetry, remote APIs, or external server dependencies, AURIX marries **bare-metal systems programming in Rust** with **modern deep learning (PyTorch, Unsloth, Qwen 3:4B)** and a **reactive GUI (Slint)**.

ZERO-SOCKET **Air-Gapped Privacy**
Zero outbound HTTP sockets, WebSockets, or cloud endpoints. Complete confidentiality for terminal commands, source code, and UI state.

GOVERNOR **Hard Resource Bounds**
Real-time OS monitoring suspends model operations if RAM exceeds 12.0 GB or VRAM exceeds 6.0 GB, preventing system lockups.

SELF-HEALING **Autonomous Recovery**
Automatic regex traceback parsing, QLoRA error diagnosis, candidate patch generation, and strict N=3 retry mitigation ceiling.

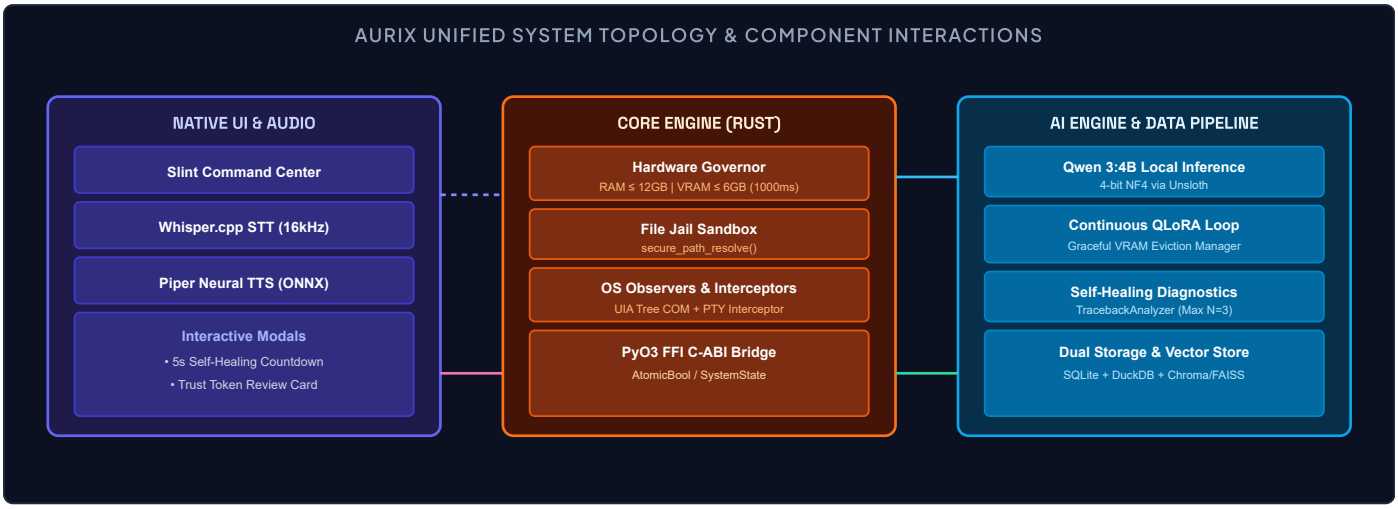
Table of Contents & Blueprint Navigation

Core System Invariants

Invariant	Implementation Mechanism	Architectural Purpose
In-Process FFI Execution	PyO3 C-ABI native extension module (`core_engine.pyd`)	Zero IPC overhead, zero localhost TCP ports, atomic memory sharing across Rust and Python runtimes.
Hard Resource Ceilings	Rust `sysinfo` + `nvml-wrapper` background thread (1000ms poll)	Strictly prevents system out-of-memory (OOM) crashes by enforcing 12.0 GB RAM and 6.0 GB VRAM limits.
Path Canonicalization Jail	Rust `fs::canonicalize` & boundary prefix verification	Mitigates path traversal attacks (`../`), symlink escapes, UNC paths, and unauthorized disk writes.
Continuous QLoRA Learning	Unsloth 4-bit NF4 fine-tuning on live telemetry streams	Student model continuously adapts to user workflows locally without full parameter recomputation.
Autonomous Self-Healing Loop	Regex traceback parser + LLM patch generation (Max N=3)	Automatically repairs failed terminal commands with a 5-second UI countdown override for user intervention.

2 Master System Architecture

AURIX employs a **tri-tier modular architecture** where each tier maintains strict separation of concerns, communicating via zero-overhead native bindings, shared atomic memory, and embedded database layers.



Runtime Subsystem Matrix

Module Name	Primary Language	Key Crates / Libraries	Core Responsibility
`core_engine`	RUST 2021	`pyo3`, `sysinfo`, `nvml-wrapper`, `uiautomation`	Hardware governor, security sandbox, Windows UI Automation tree, headless PTY execution.
`ai_engine`	PYTHON 3.10+	`torch`, `unsloth`, `transformers`, `trl`	Local 4-bit LLM generation, QLoRA continuous training loop, graceful VRAM memory manager.
`data_pipeline`	PYTHON / SQL	`sqlite3`, `duckdb`, `chromadb`, `faiss`	Telemetry ingestion daemon, OLAP analytics, self-healing traceback parser, semantic vector store.
`native_ui`	RUST / SLINT	`slint 1.8`, `windows-sys`, `piper`, `whisper`	GPU-accelerated desktop dashboard, speech-to-text, text-to-speech, security review cards.

3 Subsystem 1: Rust Core Engine — core_engine

The `core_engine` is the foundation of AURIX, compiled as a native shared library (`cdylib`) and imported directly by Python via **PyO3**. It executes bare-metal system tasks where memory safety, low latency, and direct OS APIs are essential.

1. Hardware Resource Governor (governor)

The Hardware Resource Governor is a dedicated watchdog thread designed to operate reliably on resource-constrained machines (e.g., 16 GB RAM / 8 GB VRAM). It continuously queries system metrics to protect the desktop environment from being starved by heavy machine learning operations.

RAM CEILING **12.0 GiB (75% Utilization)**
Monitored via `sysinfo::System`. If active system RAM crosses 12.0 GB, the governor instantly signals the Python QLoRA loop to halt.

VRAM CEILING **6.0 GiB (75% Utilization)**
Monitored via `nvml-wrapper` (NVIDIA Management Library). Protects device 0 VRAM from out-of-memory kernel panics.

Lock-Free Atomic Signaling (governor/atomic_state.rs)

To eliminate GIL contention and avoid network sockets, the governor writes to an `Arc<AtomicBool>` using sequentially consistent memory ordering (`Ordering::SeqCst`). This provides instantaneous cross-thread signaling with zero allocation overhead.

```
// core_engine/src/governor/atomic_state.rs
static GLOBAL_SUSPEND_FLAG: AtomicBool = AtomicBool::new(false);
pub fn set_suspend_flag(suspend: bool) { GLOBAL_SUSPEND_FLAG.store(suspend, Ordering::SeqCst); }

#[pyclass]
#[derive(Clone)]
pub struct SystemState { is_suspended: Arc<AtomicBool> }
#[pymethods]
impl SystemState {
    pub fn check_suspended(&self) -> bool { self.is_suspended.load(Ordering::SeqCst) }
}
```

2. Security Sandbox & File Jail (sandbox/file_jail.rs)

Autonomous agents must never execute uncontrolled file writes. The **File Jail** enforces path canonicalization on every path requested by the LLM before any file descriptor is opened.

> **Canonicalization Security Protocol**
> If a requested file path resolves outside the designated `ALLOWED_ROOT` (e.g. `C:\Windows\System32` or via `..\..\` traversal), `secure_path_resolve()` triggers an immediate safety panic, which PyO3 catches and translates into a Python `RuntimeError("Access Denied")`.

```
// Canonicalization & Containment Check Algorithm
pub fn secure_path_resolve(base_dir: &Path, target: &str) -> Result<PathBuf, io::Error> {
    let canon_base = fs::canonicalize(base_dir)?;
    let candidate = if Path::new(target).is_absolute() { PathBuf::from(target) } else { base_dir.join(target) };
    let canon_candidate = fs::canonicalize(&candidate)
        .or_else(|_| candidate.parent().unwrap().canonicalize().map(|p| p.join(candidate.file_name().unwrap())))?;
    if !canon_candidate.starts_with(&canon_base) {
        panic!("Access Denied: path {:?} escapes jail boundary {:?}", canon_candidate, canon_base);
    }
    Ok(canon_candidate)
}
```

3. Operating System Observers (observers)

- **Windows UI Automation COM Observer (uia_tree.rs):** Captures focused control, window class name, automation ID, and screen-space pixel coordinates without taking invasive screen captures.
- **PTY Child Process Interceptor (terminal_hook.rs):** Spawns sandboxed shell commands using `cmd.exe /C` with `CREATE_NO_WINDOW (0x08000000)` flags, capturing stdout, stderr, and exit codes.

4 Subsystem 2: AI Engine & Continuous Learning — ai_engine

The `ai_engine` executes local language model inference and continuous background fine-tuning. It is specifically architected to run alongside the Rust governor with **deterministic VRAM governance**.

1. Qwen 3:4B Local Inference Engine (llm_inference.py)

AURIX utilizes the **Qwen 3:4B / Qwen 2.5 3B** model family, loaded in **4-bit NormalFloat (NF4)** quantization via `bitsandbytes` and `Unsloth`. This reduces base model memory footprint from ~10 GB in FP16 to ~2.5 GB in VRAM, leaving ample room for context caches.

```
# Prompt Template Structure with Multi-Modal Context Grounding
PROMPT_TEMPLATE = """### Instruction:
{instruction}

### Input:
{input}

### Response:
{output}"""
```

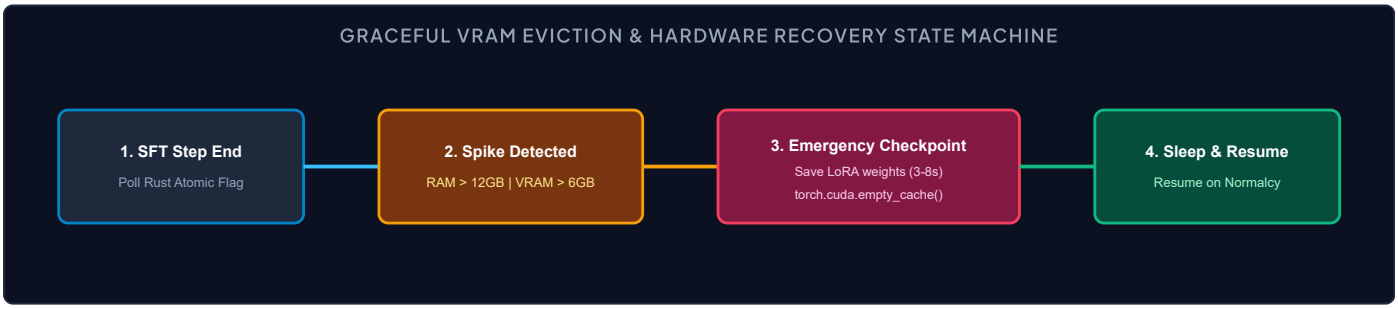
2. Unsloth QLoRA Continuous Training Loop (training/qlora_loop.py)

AURIX continuously fine-tunes a student adapter on user execution traces. To prevent GPU out-of-memory errors on an 8 GB RTX 4060, the training loop uses a tightly constrained hyperparameter profile:

Parameter	Value	Rationale & VRAM Impact
Micro-Batch Size	`1`	Minimizes peak VRAM consumption per forward/backward pass.
Gradient Accumulation	`8`	Yields an effective batch size of 8 without increasing active VRAM.
LoRA Rank (r) / Alpha	`r=16, alpha=32`	Targets all attention + MLP linear layers (`q, k, v, o, gate, up, down`).
Gradient Checkpointing	`"unsloth"`	Recomputes activations during backward pass, saving ~40% VRAM.
Optimiser	`"adamw_8bit"`	Halves optimizer state memory from 8 bytes/param to 4 bytes/param.

3. Graceful VRAM Eviction Manager (training/memory_manager.py)

The `GracefulMemoryManager` bridges the PyTorch training loop with the Rust hardware governor. At every training step, the `GovernorCallback` queries `aurix_core.SystemState.check_suspended()`.



Mechanical HDD Checkpoint Latency Mitigation

On mechanical hard drives, flushing LoRA weights blocks I/O for 3 to 8 seconds. `GracefulMemoryManager` decouples checkpoint writes from the active GUI thread to keep the interface responsive during emergency evictions.

5 Subsystem 3: Data Pipeline & Self-Healing — data_pipeline

The `data_pipeline` provides structured telemetry logging, analytical query processing, experience replay buffering, and autonomous error recovery.

1. Dual Storage Engine: SQLite + DuckDB (storage)

AURIX pairs **SQLite** for concurrent transaction logging with **DuckDB** for high-speed in-memory OLAP analytics.

Table Name	Primary Columns	Operational Purpose
`execution_logs`	`log_id, session_id, timestamp, action_type, target_command, status, return_code, error_traceback`	Tracks every terminal command and UI action. Fed into the self-healing engine upon failure.
`performance_telemetry`	`log_id, timestamp, ram_allocated_gb, vram_peak_gb, training_state`	Time-series metrics recorded at 1-second intervals from the Rust governor.
`security_audit_trails`	`audit_id, timestamp, requested_path, operation_type, trust_token_id, approval_status, sandbox_enforced`	Immutable audit log of all sandbox interactions and Trust Token approvals.

2. Autonomous Self-Healing Diagnostic Engine (self_healing)

When a terminal command returns a non-zero exit code, the **Self-Healing Engine** triggers an automated recovery workflow:

1. **Traceback Parsing:** `TracebackAnalyzer.parse_stderr()` scans backward through the stderr stream using regex to locate core exception type, message, and offending source line.
2. **Diagnostic Context Generation:** A specialized diagnostic prompt is synthesized containing the original command, error diagnosis, and traceback.
3. **Candidate Patch Synthesis:** The local Qwen 3:4B model generates a corrected script.
4. **Strict N=3 Retry Ceiling:** To prevent infinite self-healing loops, the engine caps automated retry attempts at 3. Once exceeded, control yields to the user.
5. **5-Second Override Modal:** The proposed patch and countdown are sent to the Slint GUI, allowing the user to cancel or modify the fix before execution.

```
# Self-Healing Execution Failure Handler
def handle_execution_failure(self, task_id: str, failed_script: str, stderr_stream: str) → Dict[str, Any]:
    current_attempts = self.retry_tracker.get(task_id, 0)
    if current_attempts ⩾ self.MAX_RETRIES:
        return {"status": "HALTED_MAX_RETRIES_EXCEEDED", "show_alert_modal": True}
    diagnostic = TracebackAnalyzer.parse_stderr(stderr_stream)
    prompt = self._construct_diagnostic_prompt(failed_script, diagnostic)
    proposed_patch = self._query_model_for_patch(prompt)
    self.retry_tracker[task_id] = current_attempts + 1
    return {
        "status": "PROPOSED_PATCH_READY",
        "attempt": self.retry_tracker[task_id],
        "proposed_patch": proposed_patch,
        "countdown_seconds": 5
    }
```

3. Semantic Compiler & Skill Vector Store (vector_store & compiler)

- **Semantic Compiler:** Automatically detects and parameterizes hardcoded ports (e.g. `--port=3000` -> `${PORT_NUMBER_1}`) and directory paths into generalized automation scripts.
- **Experience Replay Buffer:** Samples training batches with an **80/20 ratio** (80% foundational coding knowledge, 20% live user experience) to prevent catastrophic forgetting.
- **Skill Vector Store:** Uses **ChromaDB** for persistent cosine semantic retrieval and **FAISS** `IndexFlatL2` for microsecond bare-metal L2 vector search over 384-dimensional embeddings.

6 Subsystem 4: Native Command Center & Audio — native_ui

The `native_ui` subsystem provides a responsive desktop user interface and zero-cloud voice interaction. Built with **Slint 1.8**, it renders natively on the GPU with minimal memory overhead.

1. Slint Declarative Command Center (ui)

The interface features a cyber-industrial dark theme with real-time gauges and security controls:

TELEMETRY DIALS **Live Hardware Gauges**
Real-time SVG arc meters displaying CPU load, RAM allocation, GPU VRAM peak, and active process state.

REVIEW CARDS **Trust Token Validation**
Interactive cards requiring user approval before executing high-risk file modifications or system commands.

ALERT MODALS **5s Self-Healing Timer**
Floating alert modal displaying failed command diffs, error tracebacks, and an animated 5-second countdown timer.

2. Offline Audio Subsystem: Whisper STT & Piper TTS (audio)

SPEECH-TO-TEXT **Whisper.cpp (whisper_stt.rs)**
Captures 16 kHz 16-bit mono PCM microphone streams and performs offline GGML transcription. Dispatches recognized voice commands directly into the AURIX event bus without cloud latency.

TEXT-TO-SPEECH **Piper Neural TTS (piper_tts.rs)**
Synthesizes responses using local ONNX voice models (e.g. `en_US-lessac-medium.onnx`). Streams audio directly to the default audio output with sub-50ms synthesis latency.

```
// Slint UI Architecture — Telemetry & Review Card Binding
export component AurixCommandCenter inherits Window {
    in-out property <float> cpu_usage: 0.15;
    in-out property <string> ram_display: "8.2 / 16.0 GB";
    in-out property <string> toast_message: "";
    callback send_message(string);
    callback review_approve();
    callback alert_retry();
}
```

3. Dual-Runtime Architecture

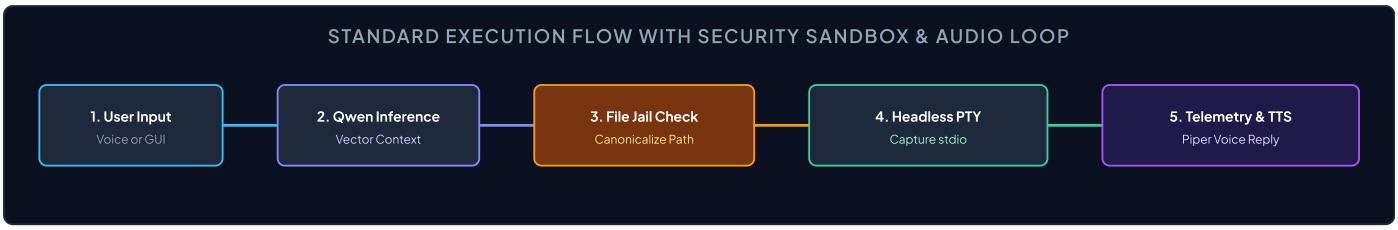
AURIX supports two unified entrypoints:

- **Python Controller (`run_ui.py`):** Orchestrates Slint UI lifecycle, connects background Qwen LLM inference queues, runs 1-second telemetry timers, and handles interactive callback bindings.
- **Native Rust Binary (`main.rs`):** Standalone bare-metal execution profile linking Slint with direct Win32 native APIs (`GlobalMemoryStatusEx`, `GetSystemTimes`).

7 End-to-End Operational Workflows

The following workflows illustrate how AURIX's subsystems coordinate across real-world operational scenarios.

Workflow A: Voice / Text Command to Sandboxed Execution



Workflow B: Autonomous Self-Healing on Execution Failure

1. Non-Zero Exit & Intercept

`terminal_hook.rs` detects `exit_code ≠ 0`. Raw stderr is piped to the `SelfHealingHook`.

2. Diagnosis & Patch Synthesis

`TracebackAnalyzer` isolates exception. Qwen 3:4B synthesizes a corrected script candidate.

3. 5s Countdown & Retest

Slint dashboard displays animated 5s modal. If uncancelled, patch executes (capped at N=3 retries).

Workflow C: Hardware Overload & Graceful QLoRA Eviction

1. Hardware Ceiling Breach

Rust monitor thread detects RAM > 12 GB or VRAM > 6 GB, writing `true` to `GLOBAL_SUSPEND_FLAG`.

2. VRAM Cache Purge

`GovernorCallback` triggers `model.save_pretrained()`, runs `gc.collect()`, and clears CUDA cache.

3. Spin-Wait & Resumption

Training thread sleeps until resource levels return to safe limits, then resumes SFT loop.

8 Configuration, Build & Deployment Guide

AURIX is configured via a centralized, human-readable TOML configuration file (`config.toml`) located in the project root.

config.toml Parameter Reference

```
# AURIX Desktop AI Agent — config.toml Specification
[security]
allowed_project_paths = ["G:/Websites By Ai/AURIX", "C:/Users/NAC/Documents/University/Projects"]
file_jail_enabled = true; read_only_mode = false; trust_token_required = true

[resources]
max_ram_gb = 12.0              # Host RAM ceiling before governor triggers suspension
max_vram_gb = 6.0              # Discrete GPU VRAM ceiling (NVIDIA RTX 4060)
cpu_throttle_percent = 85.0    # CPU usage throttle threshold
poll_interval_ms = 1000        # Watchdog polling cadence
suspend_on_overload = true

[audio]
microphone = "Default"; whisper_model_path = "models/whisper/ggml-base.en.bin"; whisper_threads = 4
piper_model_path = "models/piper/en_US-lessac-medium.onnx"; piper_config_path = "models/piper/en_US-lessac-medium.onnx.json"
auto_tts_reply = true

[llm]
model_name = "unsloth/Qwen3-4B"; max_seq_length = 2048; load_in_4bit = true; quantization = "nf4"; device = "cuda"
temperature = 0.7; top_p = 0.9
```

Build & Execution Instructions

1. Compile Core Rust Engine

```
cd core_engine
maturin develop --release
```

2. Launch Slint GUI

```
python native_ui/run_ui.py
# Or cargo run --manifest-path native_ui/Cargo.toml
```

3. Continuous QLoRA Training

```
python ai_engine/training/qlora_loop.py
```

4. Verification Test Suite

```
pytest tests/
cargo test --workspace
```

System Readiness Verification

The test suite in `tests/` validates all subsystems in isolation and end-to-end: Qwen prompt formatting (`test_qwen_model.py`), governor tripwires (`test_governor_limits.rs`), file jail canonicalization (`test_file_jail.rs`), and data pipeline integration (`test_data_pipeline.py`).